

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON
ACADEMIC MANAGEMENT SYSTEM

P R O J E C T R E P O R T

Submitted by

UDISHKRISHNA V

25ITR165

JAVA ASSIGNMENT

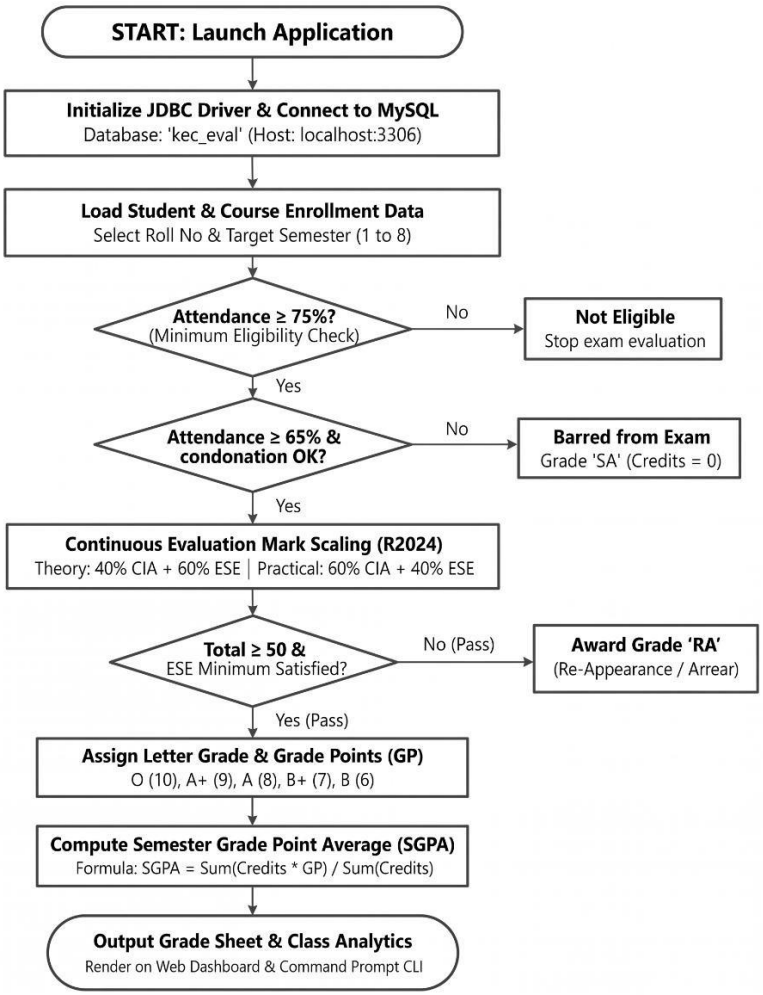
AIM:

Development of Kongu Engineering College (R2024) Student Academic & Continuous Evaluation System using Java JDBC.

OBJECTIVE:

Design and develop a database-driven Java application using JDBC to manage student records, perform incremental marks entry (Continuous Internal Assessments and End Semester Examinations), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024. System Requirements & Specifications.

FLOW CHART:



PROCEDURE:

- Load the MySQL Connector/J driver to enable communication between the Java application and the MySQL database.

- Establish a connection to the kec_eval database running on the local MySQL server using DriverManager.getConnection().

- Store and retrieve student details, including roll number, name, department, and semester, along with course information such as course code, title, credits, and course type.

- Enroll students in semester courses, record the number of classes held and attended, and calculate the attendance percentage.

- Verify examination eligibility based on attendance percentage. Students with 75% or above are eligible, those with 65%–74% require medical condonation verification, and those below 65% are marked as Shortage of Attendance (SA).

- Record CIA-1, CIA-2, and assignment marks, and calculate the scaled internal assessment marks according to the specified theory and practical course weightages.

- Record end-semester examination marks out of 100, verify the minimum passing requirements, and scale the marks according to the applicable theory and practical .

- Calculate the total marks by combining scaled CIA and ESE marks. Assign letter grades and grade points according to the specified grading scale, marking students as RA if they fail the examination or score below the required passing mark.

- Generate student grade sheets, attendance shortage reports, and department-wise pass percentage reports. Display the results through the interactive web dashboard and the command prompt console**.**

DATABASE SCHEMA:

1.STUDENTS TABLE:

```
mysql> describe students;
+------------------+-------------+------+-----+-------------------+-------------------+
| Field            | Type        | Null | Key | Default           | Extra             |
+------------------+-------------+------+-----+-------------------+-------------------+
| roll_no          | varchar(20) | NO   | PRI | NULL              |                   |
| name             | varchar(100)| NO   |     | NULL              |                   |
| department       | varchar(20) | NO   |     | NULL              |                   |
| current_semester | int         | NO   |     | 3                 |                   |
| created_at       | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+------------------+-------------+------+-----+-------------------+-------------------+
5 rows in set (0.03 sec)
```

2. COURSES TABLE:

```
mysql> describe courses;
+-------------+-----------------------------------+------+-----+-------------------+-------------------+
| Field       | Type                              | Null | Key | Default           | Extra             |
+-------------+-----------------------------------+------+-----+-------------------+-------------------+
| course_code | varchar(20)                       | NO   | PRI | NULL              |                   |
| title       | varchar(150)                      | NO   |     | NULL              |                   |
| credits     | int                               | NO   |     | NULL              |                   |
| course_type | enum('THEORY','PRACTICAL','EMBEDDED') | NO   |     | NULL              |                   |
| semester    | int                               | NO   |     | 3                 |                   |
| created_at  | timestamp                         | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+-----------------------------------+------+-----+-------------------+-------------------+
6 rows in set (0.01 sec)
```

3. ENROLLMENTS TABLE:

```
mysql> describe enrollments;
+------------------+-------------+------+-----+---------+----------------+
| Field            | Type        | Null | Key | Default | Extra          |
+------------------+-------------+------+-----+---------+----------------+
| enrollment_id    | int         | NO   | PRI | NULL    | auto_increment |
| roll_no          | varchar(20) | NO   | MUL | NULL    |                |
| course_code      | varchar(20) | NO   | MUL | NULL    |                |
| semester         | int         | NO   |     | 3       |                |
| total_classes    | int         | NO   |     | 0       |                |
| attended_classes | int         | NO   |     | 0       |                |
+------------------+-------------+------+-----+---------+----------------+
6 rows in set (0.02 sec)
```

4. ASSESSMENT MARKS TABLE:

```
mysql> describe assessment_marks;
+--------------------+--------------+------+-----+---------+----------------+
| Field              | Type         | Null | Key | Default | Extra          |
+--------------------+--------------+------+-----+---------+----------------+
| marks_id           | int          | NO   | PRI | NULL    | auto_increment |
| enrollment_id      | int          | NO   | UNI | NULL    |                |
| cia1               | decimal(5,2) | YES  |     | NULL    |                |
| cia2               | decimal(5,2) | YES  |     | NULL    |                |
| assignment_quiz_lab | decimal(5,2) | YES  |     | NULL    |                |
| ese_marks          | decimal(5,2) | YES  |     | NULL    |                |
| ese_absent         | tinyint(1)   | NO   |     | 0       |                |
+--------------------+--------------+------+-----+---------+----------------+
7 rows in set (0.02 sec)
```

SOURCE CODE:

MAINAPP.JAVA:

```
import java.sql.Connection;
import java.sql.SQLException;
import java.util.InputMismatchException;
import java.util.Scanner;

public class MainApp {

    private static final Scanner    scanner     = new Scanner(System.in);
    private static final StudentDAO studentDAO  = new StudentDAO();
    private static final AcademicDAO academicDAO = new AcademicDAO();

    public static void main(String[] args) {
        printBanner();
        testDBConnection();

        boolean running = true;
        while (running) {
            printMainMenu();
            int choice = readInt("Enter your choice: ");
            switch (choice) {
                case 1  -> menuAddStudent();
                case 2  -> menuListStudents();
                case 3  -> menuAddCourse();
                case 4  -> menuListCourses();
                case 5  -> menuEnrollStudent();
                case 6  -> menuUpdateAttendance();
                case 7  -> menuShowEnrollments();

                case 8  -> menuEnterPhase1CIA();
                case 9  -> menuEnterPhase2Assignment();
                case 10 -> menuEnterPhase3ESE();
                case 11 -> menuViewCurrentMarks();

                case 12 -> menuMarkSheet();
                case 13 -> academicDAO.printAttendanceDefaulterReport();
```

```
            case 14 -> academicDAO.printClassPerformanceSummary();

            case 0  -> {
               System.out.println("\n  Closing application. Goodbye!");
               DBConnection.closeConnection();
               running = false;
            }
            default -> System.out.println("  [WARN] Invalid choice. Please try again.");
          }
       }
       scanner.close();
    }

    private static void printBanner() {
       System.out.println();
       System.out.println("*".repeat(70));
       System.out.println("*" + " ".repeat(68) + "*");
       System.out.println("*    KONGU ENGINEERING COLLEGE (AUTONOMOUS)
*");
       System.out.println("*    Student Academic & Continuous Evaluation System
*");
       System.out.println("*    Regulation 2024  |  Java JDBC Application                *");
       System.out.println("*" + " ".repeat(68) + "*");
       System.out.println("*".repeat(70));
    }

    private static void printMainMenu() {
       System.out.println("\n" + "=".repeat(55));
       System.out.println("            MAIN MENU");
       System.out.println("=".repeat(55));
       System.out.println("  -- Student & Course Management --");
       System.out.println("   1.  Add New Student");
       System.out.println("   2.  List All Students");
       System.out.println("   3.  Add New Course");
       System.out.println("   4.  List All Courses");
       System.out.println("   5.  Enroll Student in Course");
       System.out.println("   6.  Update Attendance");
       System.out.println("   7.  View Student Enrollments & Attendance");
       System.out.println();
       System.out.println("  -- Incremental Marks Entry (Phase-wise) --");
       System.out.println("   8.  Phase 1 – Enter CIA Test 1 & CIA Test 2");
       System.out.println("   9.  Phase 2 – Enter Assignment / Quiz / Lab");
       System.out.println("  10.  Phase 3 – Enter ESE Marks / Mark Absent");
       System.out.println("  11.  View Current Marks for Enrollment");
       System.out.println();
       System.out.println("  -- Reports --");
       System.out.println("  12.  Student Semester Mark Sheet");
       System.out.println("  13.  Attendance Defaulter Report  (SA Grade)");
       System.out.println("  14.  Class Performance & Result Summary");
       System.out.println();
       System.out.println("   0.  Exit");
       System.out.println("=".repeat(55));
    }

    private static void menuAddStudent() {
```

```
        System.out.println("\n--- Add New Student ---");
        System.out.print("  Roll Number (e.g., 24ITR001) : ");
        String roll = scanner.nextLine().trim().toUpperCase();
        System.out.print("  Student Name                 : ");
        String name = scanner.nextLine().trim();
        System.out.print("  Department (CSE/IT/ECE/…)    : ");
        String dept = scanner.nextLine().trim().toUpperCase();
        int sem = readInt("  Current Semester (1-8)       : ");

        if (roll.isEmpty() || name.isEmpty() || dept.isEmpty()) {
            System.out.println("  [ERROR] Fields cannot be blank.");
            return;
        }
        if (sem < 1 || sem > 8) {
            System.out.println("  [ERROR] Semester must be between 1 and 8.");
            return;
        }

        boolean ok = studentDAO.addStudent(roll, name, dept, sem);
        if (ok) System.out.println("  [SUCCESS] Student '" + roll + "' added successfully.");
    }

    private static void menuListStudents() {
        studentDAO.listAllStudents();
    }

    private static void menuAddCourse() {
        System.out.println("\n--- Add New Course ---");
        System.out.print("  Course Code  (e.g., 24ITR31)         : ");
        String code = scanner.nextLine().trim().toUpperCase();
        System.out.print("  Course Title                         : ");
        String title = scanner.nextLine().trim();
        int credits = readInt("  Credits (1-6)                        : ");
        System.out.println("  Course Type options: THEORY | PRACTICAL | EMBEDDED");
        System.out.print("  Course Type                          : ");
        String type = scanner.nextLine().trim().toUpperCase();

        if (code.isEmpty() || title.isEmpty()) {
            System.out.println("  [ERROR] Code and title cannot be blank.");
            return;
        }
        if (credits < 1 || credits > 6) {
            System.out.println("  [ERROR] Credits must be between 1 and 6.");
            return;
        }
        if (!type.equals("THEORY") && !type.equals("PRACTICAL") &&
!type.equals("EMBEDDED")) {
            System.out.println("  [ERROR] Course type must be THEORY, PRACTICAL, or
EMBEDDED.");
            return;
        }

        boolean ok = studentDAO.addCourse(code, title, credits, type);
        if (ok) System.out.println("  [SUCCESS] Course '" + code + "' added successfully.");
    }
```

```
    private static void menuListCourses() {
        studentDAO.listAllCourses();
    }

    private static void menuEnrollStudent() {
        System.out.println("\n--- Enroll Student in Course ---");
        System.out.print("  Roll Number  : ");
        String roll = scanner.nextLine().trim().toUpperCase();
        System.out.print("  Course Code  : ");
        String code = scanner.nextLine().trim().toUpperCase();
        int sem = readInt("  Semester     : ");

        if (!studentDAO.studentExists(roll)) {
            System.out.println("  [ERROR] Student '" + roll + "' not found.");
            return;
        }
        if (!studentDAO.courseExists(code)) {
            System.out.println("  [ERROR] Course '" + code + "' not found.");
            return;
        }

        boolean ok = studentDAO.enrollStudent(roll, code, sem);
        if (ok) System.out.println("  [SUCCESS] " + roll + " enrolled in " + code + " for
Semester " + sem + ".");
    }

    private static void menuUpdateAttendance() {
        System.out.println("\n--- Update Attendance ---");
        System.out.print("  Roll Number  : ");
        String roll = scanner.nextLine().trim().toUpperCase();
        System.out.print("  Course Code  : ");
        String code = scanner.nextLine().trim().toUpperCase();
        int sem = readInt("  Semester     : ");

        int enrollmentId = studentDAO.getEnrollmentId(roll, code, sem);
        if (enrollmentId == -1) {
            System.out.println("  [ERROR] Enrollment not found for the given combination.");
            return;
        }

        System.out.println("  Choose mode:");
        System.out.println("    1. SET total attendance (overwrite)");
        System.out.println("    2. ADD incremental attendance (delta)");
        int mode = readInt("  Mode: ");

        if (mode == 1) {
            int held = readInt("  Total classes HELD    : ");
            int att  = readInt("  Total classes ATTENDED: ");
            if (att > held) { System.out.println("  [ERROR] Attended cannot exceed held.");
return; }
            boolean ok = studentDAO.setAttendance(enrollmentId, held, att);
            if (ok) System.out.println("  [SUCCESS] Attendance set: " + att + "/" + held);
        } else if (mode == 2) {
            int deltaHeld = readInt("  Additional classes HELD     : ");
```

```
            int deltaAtt  = readInt("  Additional classes ATTENDED : ");
            if (deltaAtt > deltaHeld) { System.out.println("  [ERROR] Attended cannot exceed
held."); return; }
            boolean ok = studentDAO.updateAttendance(enrollmentId, deltaHeld, deltaAtt);
            if (ok) System.out.println("  [SUCCESS] Attendance incremented by " + deltaAtt + "/"
+ deltaHeld + ".");
        } else {
            System.out.println("  [ERROR] Invalid mode selection.");
        }
    }

    private static void menuShowEnrollments() {
        System.out.println("\n--- View Student Enrollments ---");
        System.out.print("  Roll Number: ");
        String roll = scanner.nextLine().trim().toUpperCase();
        if (!studentDAO.studentExists(roll)) {
            System.out.println("  [ERROR] Student '" + roll + "' not found.");
            return;
        }
        studentDAO.showStudentEnrollments(roll);
    }

    private static void menuEnterPhase1CIA() {
        System.out.println("\n--- Phase 1: Enter CIA Test 1 & CIA Test 2 ---");
        int eid = getEnrollmentIdFromUser();
        if (eid == -1) return;

        double cia1 = readDouble("  CIA Test 1 Marks (0-100): ");
        double cia2 = readDouble("  CIA Test 2 Marks (0-100): ");

        boolean ok = academicDAO.enterPhase1CIA(eid, cia1, cia2);
        if (ok) System.out.println("  [SUCCESS] CIA 1 & CIA 2 marks saved.");
        academicDAO.showCurrentMarks(eid);
    }

    private static void menuEnterPhase2Assignment() {
        System.out.println("\n--- Phase 2: Enter Assignment / Quiz / Lab Marks ---");
        int eid = getEnrollmentIdFromUser();
        if (eid == -1) return;

        double aql = readDouble("  Assignment/Quiz/Lab Marks (0-100): ");

        boolean ok = academicDAO.enterPhase2Assignment(eid, aql);
        if (ok) System.out.println("  [SUCCESS] Assignment/Quiz/Lab marks saved.");
        academicDAO.showCurrentMarks(eid);
    }

    private static void menuEnterPhase3ESE() {
        System.out.println("\n--- Phase 3: Enter End Semester Examination (ESE) ---");
        int eid = getEnrollmentIdFromUser();
        if (eid == -1) return;

        System.out.print("  Was student ABSENT in ESE? (yes/no): ");
        String ans = scanner.nextLine().trim().toLowerCase();
        boolean absent = ans.equals("yes") || ans.equals("y");
```

```
        double eseMarks = 0;
        if (!absent) {
            eseMarks = readDouble("  ESE Marks (0-100): ");
        }

        boolean ok = academicDAO.enterPhase3ESE(eid, eseMarks, absent);
        if (ok) System.out.println("  [SUCCESS] ESE entry saved.");
        academicDAO.showCurrentMarks(eid);
    }

    private static void menuViewCurrentMarks() {
        System.out.println("\n--- View Current Marks ---");
        int eid = getEnrollmentIdFromUser();
        if (eid == -1) return;
        academicDAO.showCurrentMarks(eid);
    }

    private static void menuMarkSheet() {
        System.out.println("\n--- Student Semester Mark Sheet ---");
        System.out.print("  Roll Number : ");
        String roll = scanner.nextLine().trim().toUpperCase();
        if (!studentDAO.studentExists(roll)) {
            System.out.println("  [ERROR] Student '" + roll + "' not found.");
            return;
        }
        int sem = readInt("  Semester    : ");
        academicDAO.printMarkSheet(roll, sem);
    }

    private static int getEnrollmentIdFromUser() {
        System.out.print("  Roll Number : ");
        String roll = scanner.nextLine().trim().toUpperCase();
        System.out.print("  Course Code : ");
        String code = scanner.nextLine().trim().toUpperCase();
        int sem = readInt("  Semester    : ");

        int eid = studentDAO.getEnrollmentId(roll, code, sem);
        if (eid == -1) {
            System.out.println("  [ERROR] No enrollment found for " + roll + " / " + code + " /
Sem " + sem + ".");
            System.out.println("          Please verify student is enrolled first (Option 5).");
        }
        return eid;
    }

    private static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            String line = scanner.nextLine().trim();
            try {
                return Integer.parseInt(line);
            } catch (NumberFormatException e) {
                System.out.println("  [ERROR] Please enter a valid integer.");
            }
```

```
        }
    }

    private static double readDouble(String prompt) {
        while (true) {
            System.out.print(prompt);
            String line = scanner.nextLine().trim();
            try {
                return Double.parseDouble(line);
            } catch (NumberFormatException e) {
                System.out.println("  [ERROR] Please enter a valid decimal number.");
            }
        }
    }

    private static void testDBConnection() {
        System.out.println("\n  Connecting to database...");
        try {
            Connection con = DBConnection.getConnection();
            if (con != null && !con.isClosed()) {
                System.out.println("  [OK] Connected successfully! Engine: " +
DBConnection.getActiveEngine());
            }
        } catch (SQLException e) {
            System.out.println("\n" + "!".repeat(60));
            System.out.println("  DATABASE CONNECTION FAILED!");
            System.out.println("  " + e.getMessage());
            System.out.println("!".repeat(60));
            System.out.println("\n  Steps to fix:");
            System.out.println("  1. Ensure MySQL server is running.");
            System.out.println("  2. Run kec_schema.sql in MySQL Workbench or CLI.");
            System.out.println("  3. Update DB credentials in DBConnection.java.");
            System.out.println("  4. Add mysql-connector-j-*.jar to your classpath.");
            System.out.println("\n  Exiting...");
            System.exit(1);
        }
    }
}
```

DB CONNECTION.JAVA:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL      =
"jdbc:mysql://localhost:3306/kec_academic?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=A
sia/Kolkata";
    private static final String USER     = "root";
    private static final String PASSWORD = "root";
    private static final String DRIVER   = "com.mysql.cj.jdbc.Driver";

    private static Connection connection = null;

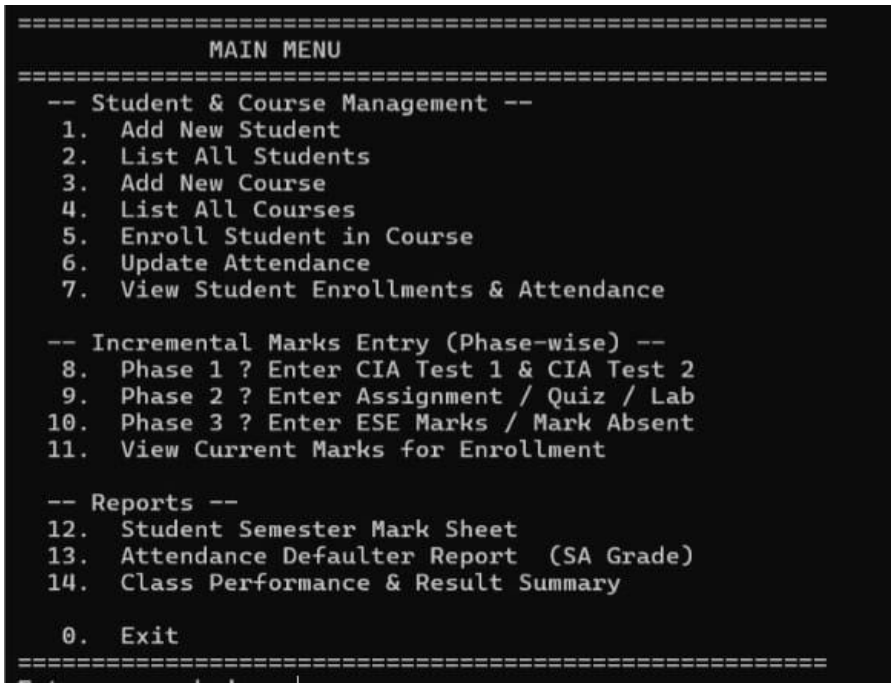
    private DBConnection() {}

    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            try {
                Class.forName(DRIVER);
                connection = DriverManager.getConnection(URL, USER, PASSWORD);
                connection.setAutoCommit(true);
            } catch (ClassNotFoundException e) {
                throw new SQLException("[ERROR] MySQL JDBC Driver not found: " + e.getMessage());
            }
        }
        return connection;
    }

    public static void closeConnection() {
        if (connection != null) {
            try {
                if (!connection.isClosed()) {
                    connection.close();
                }
            } catch (SQLException e) {
                System.err.println("[WARN] Could not close connection: " + e.getMessage());
            }
        }
    }
}
```

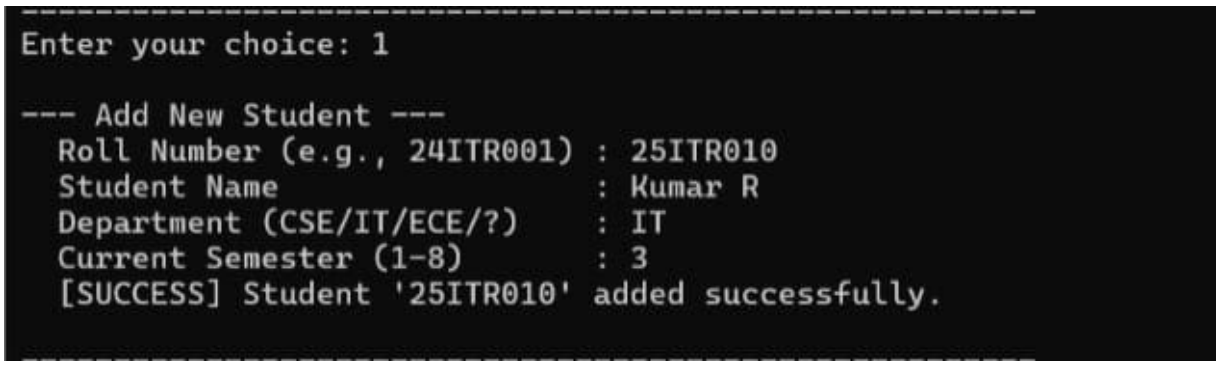
OUTPUT SCREENSHOTS:

MAIN MENU:

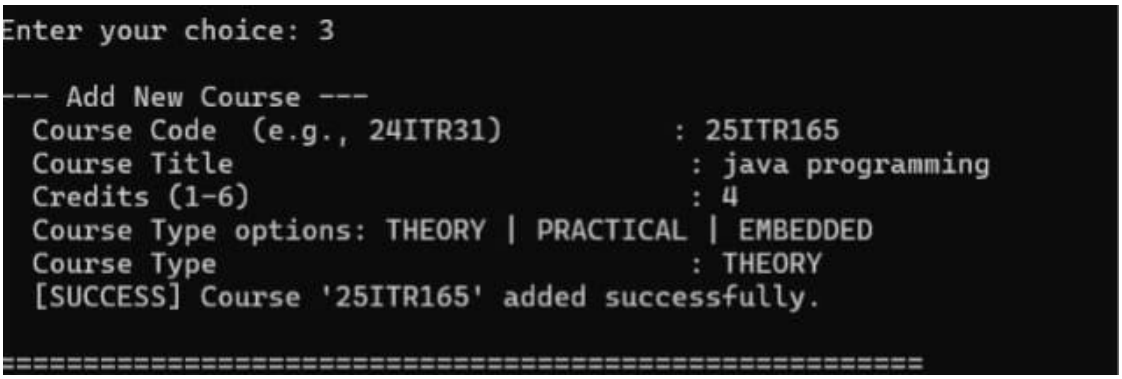


STUDENT & COURSE MANAGEMENT:

1.STUDENT REGISTRATION:



2. COURSE CATALOG:



3. COURSE ENROLLMENT & ATTENDANCE TRACKING:

```
Enter your choice: 6

--- Update Attendance ---
  Roll Number  : 24ITR010
  Course Code  : 25ITR165
  Semester     : 3
  Choose mode:
    1. SET total attendance (overwrite)
    2. ADD incremental attendance (delta)
  Mode: 1
  Total classes HELD    : 60
  Total classes ATTENDED: 54
  [SUCCESS] Attendance set: 54/60

============================================================
```

INCREMENTAL MARKS ENTRY:

1. PHASE 1 (MID-SEMESTER):

```
mysql> select marks_id, enrollment_id, cia1, cia2, assignment_quiz_lab, ese_marks from assessment_marks;
+----------+--------------+-------+-------+--------------------+-----------+
| marks_id | enrollment_id | cia1  | cia2  | assignment_quiz_lab | ese_marks |
+----------+--------------+-------+-------+--------------------+-----------+
|        1 |            1 | 88.00 | 92.00 | NULL               | NULL      |
|        2 |            2 | 75.00 | 78.00 | NULL               | NULL      |
+----------+--------------+-------+-------+--------------------+-----------+
2 rows in set (0.01 sec)
```

2. PHASE 2 (END OF SEMESTER - CIA):

```
mysql> select marks_id, enrollment_id, cia1, cia2, assignment_quiz_lab, ese_marks from assessment_marks;
+----------+--------------+-------+-------+--------------------+-----------+
| marks_id | enrollment_id | cia1  | cia2  | assignment_quiz_lab | ese_marks |
+----------+--------------+-------+-------+--------------------+-----------+
|        1 |            1 | 88.00 | 92.00 | 95.00              | NULL      |
|        2 |            2 | 75.00 | 78.00 | 80.00              | NULL      |
+----------+--------------+-------+-------+--------------------+-----------+
2 rows in set (0.01 sec)
```

3. PHASE 3 (POST SEMESTER):

```
mysql> select * from assessment_marks;
+----------+--------------+-------+-------+--------------------+-----------+------------+
| marks_id | enrollment_id | cia1  | cia2  | assignment_quiz_lab | ese_marks | ese_absent |
+----------+--------------+-------+-------+--------------------+-----------+------------+
|        1 |            1 | 88.00 | 92.00 | 95.00              | 84.00     |          0 |
|        2 |            2 | 75.00 | 78.00 | 80.00              | NULL      |          1 |
+----------+--------------+-------+-------+--------------------+-----------+------------+
2 rows in set (0.02 sec)
```

ATTENDANCE & GRADING RULES:

1.ATTENDANCE FILTER:

```
--------------------------------------------------------------------------------------------
Enter your choice (1-8): 6

Enter Semester to check (e.g., 3): 3

============================================================================================================
                    KONGU ENGINEERING COLLEGE (AUTONOMOUS) - REGULATION 2024
                         ATTENDANCE DEFAULTER & CONDONATION REPORT
                                   SEMESTER: 3
============================================================================================================
Roll No      Student Name         Dept     Course     Held     Attended Attd %     Condonation  Eligibility / Grade
------------------------------------------------------------------------------------------------------------
24ADR010     Ananya S             AIDS     24ADT31    60       32          53.3% NOT APPROVED BARRED (Grade SA)
24ADR010     Ananya S             AIDS     24ADC31    60       32          53.3% NOT APPROVED BARRED (Grade SA)
24ADR010     Ananya S             AIDS     24ADL31    30       16          53.3% NOT APPROVED BARRED (Grade SA)
24CER003     Aravind K            CIVIL    24CET31    60       42          70.0% APPROVED     CONDONED (Eligible)
24CER003     Aravind K            CIVIL    24CEC31    60       42          70.0% APPROVED     CONDONED (Eligible)
24CER003     Aravind K            CIVIL    24CEL31    30       21          70.0% NOT APPROVED PENDING (Grade SA)
------------------------------------------------------------------------------------------------------------
 Note: <65% Attendance: Shortage of Attendance (SA - Barred) | 65-74.9%: Requires Medical Condonation
============================================================================================================


Press [Enter] to return to the Main Menu...
```

STUDENT SEMESTER MARK SHEET:

```
================================================================================================
        KONGU ENGINEERING COLLEGE (AUTONOMOUS) – Affiliated to Anna University
                  STUDENT SEMESTER MARK SHEET  [Regulation 2024]
================================================================================================
  Roll Number : 24ITR001              Name       : Arun Kumar R
  Department  : IT                    Semester   : 3
------------------------------------------------------------------------------------------------
Code       Title                        Cr   Type         CIA1   CIA2   Asgn  CIA Tot ESE Sc  Gr
Status
------------------------------------------------------------------------------------------------
24ITR31    Database Management Systems  4    THEORY       78.0   82.0   88.0    32.96  43.20  A
PASS
24ITR32    Operating Systems            3    THEORY       55.0   48.0   60.0    21.62  24.00  RA
FAIL*
24ITR33    DBMS Laboratory              2    PRACTICAL    80.0   85.0   90.0    51.75  35.20  A+
PASS
------------------------------------------------------------------------------------------------

  Total Credits Enrolled  : 9
  Total Credit Points     : 50.00
  SGPA                    : 5.56
  Equivalent Percentage   : 55.56 %
  Courses Passed          : 2  |  Courses Failed : 1
  * = Failed ESE Minimum Threshold (24.00 < 27.00 required)
================================================================================================
```

DEPARTMENT WISE CLASS PERFORMANCE & RESULT SUMMARY:

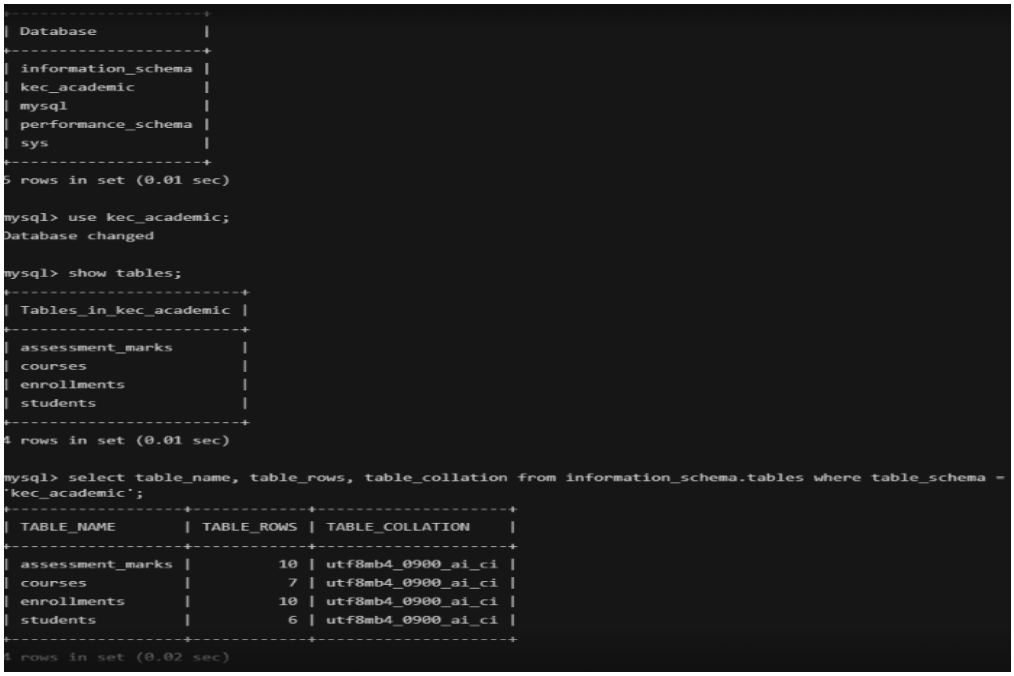
```
Department : CSE          Semester : 3
--------------------------------------------------------------------------
Roll No.      SGPA    Pct%    Pass  Fail    Status
--------------------------------------------------------------------------
24CSE001        8.33   83.3%     2       0  PASS
24CSE002        0.00    0.0%     0       1  FAIL (SA)
--------------------------------------------------------------------------
Total Students : 2      Passed : 1      Failed : 1      Pass Rate : 50%
Class Average SGPA : 4.17   Equivalent Percentage : 41.67 %
--------------------------------------------------------------------------

Department : ECE          Semester : 3
--------------------------------------------------------------------------
Roll No.      SGPA    Pct%    Pass  Fail    Status
--------------------------------------------------------------------------
24ECE001        9.00   90.0%     1       0  PASS
--------------------------------------------------------------------------
Total Students : 1      Passed : 1      Failed : 0      Pass Rate : 100%
Class Average SGPA : 9.00   Equivalent Percentage : 90.00 %
--------------------------------------------------------------------------

Department : IT           Semester : 3
--------------------------------------------------------------------------
Roll No.      SGPA    Pct%    Pass  Fail    Status
--------------------------------------------------------------------------
24ITR001        5.56   55.6%     2       1  FAIL (RA)
24ITR002        5.14   51.4%     1       1  FAIL (RA)
24ITR005        9.00   90.0%     1       0  PASS
24ITR010        9.00   90.0%     1       0  PASS
--------------------------------------------------------------------------
Total Students : 4      Passed : 2      Failed : 2      Pass Rate : 50%
Class Average SGPA : 7.17   Equivalent Percentage : 71.75 %
--------------------------------------------------------------------------
==========================================================================
```

DATABASE:

STUDENTS DATABASE:



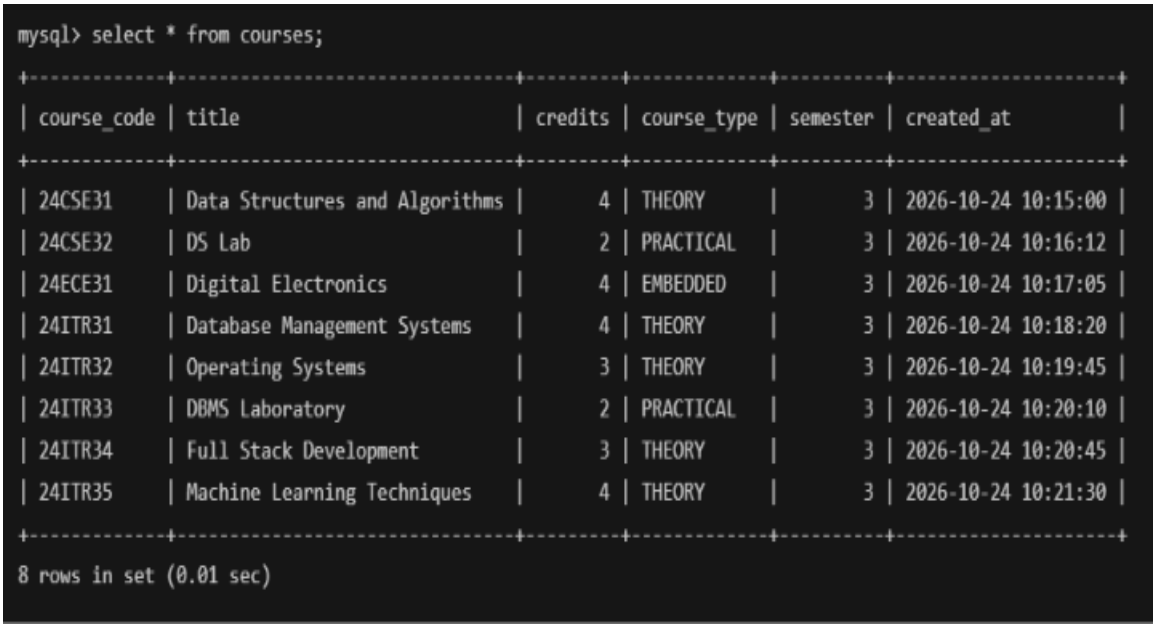
```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| kec_academic       |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.01 sec)

mysql> use kec_academic;
Database changed

mysql> show tables;
+-------------------------+
| Tables_in_kec_academic  |
+-------------------------+
| assessment_marks        |
| courses                 |
| enrollments             |
| students                |
+-------------------------+
4 rows in set (0.01 sec)

mysql> select table_name, table_rows, table_collation from information_schema.tables where table_schema =
'kec_academic';
+------------------+------------+--------------------+
| TABLE_NAME       | TABLE_ROWS | TABLE_COLLATION    |
+------------------+------------+--------------------+
| assessment_marks |         10 | utf8mb4_0900_ai_ci |
| courses          |          7 | utf8mb4_0900_ai_ci |
| enrollments      |         10 | utf8mb4_0900_ai_ci |
| students         |          6 | utf8mb4_0900_ai_ci |
+------------------+------------+--------------------+
4 rows in set (0.02 sec)
```

COURSES DATABASE:



```
mysql> select * from courses;
+-------------+--------------------------------+---------+-------------+----------+---------------------+
| course_code | title                          | credits | course_type | semester | created_at          |
+-------------+--------------------------------+---------+-------------+----------+---------------------+
| 24CSE31     | Data Structures and Algorithms |       4 | THEORY      |        3 | 2026-10-24 10:15:00 |
| 24CSE32     | DS Lab                         |       2 | PRACTICAL   |        3 | 2026-10-24 10:16:12 |
| 24ECE31     | Digital Electronics            |       4 | EMBEDDED    |        3 | 2026-10-24 10:17:05 |
| 24ITR31     | Database Management Systems    |       4 | THEORY      |        3 | 2026-10-24 10:18:20 |
| 24ITR32     | Operating Systems              |       3 | THEORY      |        3 | 2026-10-24 10:19:45 |
| 24ITR33     | DBMS Laboratory                |       2 | PRACTICAL   |        3 | 2026-10-24 10:20:10 |
| 24ITR34     | Full Stack Development         |       3 | THEORY      |        3 | 2026-10-24 10:20:45 |
| 24ITR35     | Machine Learning Techniques    |       4 | THEORY      |        3 | 2026-10-24 10:21:30 |
+-------------+--------------------------------+---------+-------------+----------+---------------------+
8 rows in set (0.01 sec)
```

ENROLLMENTS DATABASE:

```
mysql> select * from enrollments;
+---------------+----------+-------------+----------+---------------+------------------+
| enrollment_id | roll_no  | course_code | semester | total_classes | attended_classes |
+---------------+----------+-------------+----------+---------------+------------------+
|             1 | 24ITR001 | 24ITR31     |        3 |            90 |               78 |
|             2 | 24ITR001 | 24ITR32     |        3 |            70 |               52 |
|             3 | 24ITR001 | 24ITR33     |        3 |            60 |               56 |
|             4 | 24ITR002 | 24ITR31     |        3 |            90 |               80 |
|             5 | 24ITR002 | 24ITR32     |        3 |            70 |               60 |
|             6 | 24CSE001 | 24CSE31     |        3 |            80 |               75 |
|             7 | 24CSE001 | 24CSE32     |        3 |            60 |               58 |
|             8 | 24CSE002 | 24CSE31     |        3 |            80 |               50 |
|             9 | 24ECE001 | 24ECE31     |        3 |            85 |               82 |
|            10 | 24ITR005 | 24ITR34     |        3 |            50 |               45 |
|            11 | 24ITR010 | 24ITR35     |        3 |            60 |               54 |
+---------------+----------+-------------+----------+---------------+------------------+
11 rows in set (0.01 sec)
```

RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed, and tested using Java (JDBC) and MySQL. The system accurately automates the calculation of Continuous Internal Assessments (CIA), End Semester Examinations (ESE), Letter Grades, Attendance Eligibility, and Semester Grade Point Average (SGPA).

REFERENCES:

1. Oracle, Java SE Documentation – JDBC Overview and Architecture. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/ .

2. Oracle, Java SE API – HttpServer. https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html .

3. Oracle, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/ .

4. Oracle, MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/8.0/en/ .

5. University Grants Commission, Guidelines for Choice Based Credit System (CBCS). https://www.ugc.gov.in/pdfnews/8023719_Guidelines-for-CBCS.pdf .

6. Chart.js, Chart.js Documentation. https://www.chartjs.org/docs/latest/ .

7. Tailwind CSS, Tailwind CSS Documentation. https://tailwindcss.com/docs .

8. GeeksforGeeks, Establishing JDBC Connection in Java. https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/ .

9. YouTube, Java JDBC with MySQL Tutorial Videos. https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial .

10. Anthropic, Claude AI Assistant (used for project ideas and guidance). https://claude.ai/ .

GITHUB:

LINK:	https://github.com/udishkrishna447/java-project1